Interview Assignment: Privacy-First Local AI CLI Assistant

Context & Overview

We are building a local-first, privacy-preserving AI CLI assistant. This command-line agent runs directly on a user's local workstation, processing active contextual events (such as clipboard changes, shell commands, active text notes, and local document logs), extracting key long-term "memories," and allowing the user to search or query their historical context directly from the terminal.
Our core product promise is: **User data stays local by default. Cloud AI services are only leveraged with explicit, user-configured consent and robust local privacy boundaries.**
This task is designed to evaluate your system architecture judgment, desktop/CLI database design, privacy engineering intuition, threat modeling capabilities, and your ability to write clean, functional prototype code.

The Task

Build a lightweight CLI tool or local background process prototype that demonstrates a secure, privacy-preserving local AI memory pipeline. Your prototype must implement the following 6 core capabilities:

1. Ingest Mock Events

Accept raw JSON events representing historical user workstation activities. Your pipeline must support at least two distinct event types (e.g., transcript, shell_history, browser_metadata, or note).
Standard Mock Data Payload (mock_events.json):

```
[
  {
    "type": "transcript",
    "app": "Slack",
    "source": "Engineering Channel",
    "content": "Rahul said we should not expose the Stripe secret key
in our frontend. We discussed moving payment intent creation to the
backend for the pricing page.",
    "timestamp": "2026-05-19T10:30:00Z"
  },
  {
    "type": "note",
    "app": "Obsidian",
    "source": "Work Notes",
    "content": "Reviewed the Q2 marketing roadmap. Public launch is
set for June 15th. We need to draft public press releases next week.",
```

```
    "timestamp": "2026-05-19T11:15:00Z"
  },
  {
    "type": "browser_metadata",
    "app": "Chrome",
    "source": "Internal Portal",
    "content": "Viewing patient summary portal: ID 908-112-AC.
Diagnostics show elevated heart rate but stable blood pressure.",
    "timestamp": "2026-05-19T11:45:00Z"
  }
]
```

2. Extract Structured Memories

Process raw, unstructured ingested events and synthesize them into structured long-term memories.

Target Output Format:

```
{
  "people": ["Rahul"],
  "projects": ["Pricing Page"],
  "topics": ["Stripe", "Payment Intent", "Backend Security"],
  "summary": "Discussed moving Stripe payment intent creation to the
backend to avoid exposing secret keys.",
  "sensitivity": "secret"
}
```

Note: Design the pipeline execution and extraction mechanics as you see fit. You will be evaluated on your data flow separation and how cleanly you orchestrate this layer.

3. Classify Privacy Sensitivity

Analyze each raw event or extracted memory and assign it to a sensitivity tier based on the following classification rules:

Sensitivity Tier	Definition
secret	API keys, secret tokens, private keys, database passwords, credentials
confidential	Intellectual property, financial metrics, proprietary internal business details
personal	Private conversations, logs, health/medical context, identity documents
public	Publicly accessible information, general documentation, reference websites

Your classifier must return both the chosen classification tier and a **structured explanation** justifying the decision.

4. Local Storage

Persist your application state locally on the workstation.

- Implement storage for both raw historical events and extracted structured memories.
- Be prepared to defend your choice of storage engines, schema designs, and data separations in your architectural note.

5. Search Interface

Implement a simple CLI query interface to retrieve processed memories.

```
# Example interface contract
$ local-ai search "Stripe pricing"
```

Determine and implement an appropriate search strategy (indexing, keyword matching, or semantic search) that balances local CPU/memory usage on a developer's workstation with search accuracy.

6. Local-First AI Routing Decision

For any query or processing action, your CLI system must evaluate active user safety configurations and issue an explicit routing verdict.

User Policy Settings Configuration (settings.json):

```
{
  "cloud_ai_enabled": true,
  "send_personal_data_to_cloud": false,
  "send_confidential_data_to_cloud": false,
  "send_secret_data_to_cloud": false
}
```

The output of your routing engine must return a JSON response containing the calculated path (local_ai, cloud_ai, or blocked) along with the evaluation logic.

Required Deliverables

1. Prototype Codebase

Provide a repository link or archive containing clean, cross-platform runnable code with step-by-step instructions. Any mainstream language (Python, Node/TypeScript, Go, Rust) is welcome.

```
# Ensure commands to run your ingestion and search pipeline are
clearly documented:
$ [setup commands]
$ [ingestion execution command]
$ [search execution command]
```

2. Architecture & Design Note

A brief markdown document addressing your technical choices and trade-offs:

1. **Data Separation & Isolation:** Explain how you isolated raw events from high-value extracted memories, and why.
2. **Storage Rationale:** Defend your choice of storage engine for a local CLI / daemon context.
3. **Local Encryption & Key Custody:** Detail your strategy for securing and encrypting local files on a standard host OS (e.g., how you would manage decryption keys securely, interact with system keyrings/keychains, or use environment secrets securely).
4. **Data Lifecycle & Deletion:** Outline how your design would support secure, targeted deletions (e.g., "Forget everything from the last 2 hours" or a full-system wipe).

3. CLI Application Threat Model

Analyze the security posture of your local CLI assistant. For each of the following 5 target scenarios, analyze the **Risk/Impact** and propose a **Technical Mitigation Strategy** based on your architecture:

Security Threat	Risk / Technical Impact	Proposed Technical Mitigation
Host Process / File Compromise *(Malicious processes or secondary local users read raw CLI state files)*	*[Candidate to fill]*	*[Candidate to fill]*
Contextual Prompt Injection *(Ingested raw logs/clipboard events contain malicious instructions designed to hijack system prompts)*	*[Candidate to fill]*	*[Candidate to fill]*
Implicit Cloud Exfiltration *(Sensitive metadata bypasses filters or is sent during transit to cloud APIs)*	*[Candidate to fill]*	*[Candidate to fill]*
Sensitive Leakage in Diagnostics *(Sensitive PII/secrets leak into standard error outputs, terminal history, or log files)*	*[Candidate to fill]*	*[Candidate to fill]*
Unintentional Context Capture *(CLI background ingestion captures sensitive items like password generator outputs or secure environment keys)*	*[Candidate to fill]*	*[Candidate to fill]*

Scope Boundaries (What NOT to build)

Please **do not spend time on**:

- Writing actual system-level active window trackers or live keyboard sniffer loops

- Custom database or cryptographic engines
- Developing GUI clients, custom desktop dashboards, or web interfaces
- Implementing user authentication providers (SAML, OAuth) or multi-device sync engines

Evaluation Criteria

Evaluation Rubric	
Privacy & Security Thinking	30%
Architecture & Separation	25%
Smart Routing Pipeline	15%
Prototype Functionality	15%
Documentation & Code Quality	15%

Green Flags (What we look for)

- Explicit architectural boundaries between untrusted historical logs and structured memory storage.
- Smart local filtering policies that occur *before* data is serialized for network transit.
- Clear database schema layouts that accommodate data-pruning and custom-range deletions easily.
- Clean, minimalist code setups that execute without complex or heavy dependency requirements.

Red Flags

- Architectures that defer classification or sanitization entirely to a cloud service.
- Application structures where secrets, raw contexts, or raw personal data are leaked into stdout/stderr/logs.
- Making assumptions about data encryption or key custody on local machines without understanding operating system limitations.